

Personal To-Do List Application

Python Source Code Document

INTERNSHIP PROGRAM

VaultofCodes Python Internship

ASSIGNMENT

Assignment 4

STUDENT NAME

Kanak Chaudhary

PROGRAMMING LANGUAGE

Python 3

SOURCE FILE

todo.py

Document Description

This document contains the complete Python source code for the Personal To-Do List Application developed as part of the VaultofCodes Python Internship – Assignment 4. The application enables users to add, view, edit, complete, and delete tasks through a command-line interface while storing data persistently using JSON file handling.

```
"""
```

PERSONAL TO-DO LIST APPLICATION

A command-line Python application to create, view, edit, mark complete, and delete tasks — organized by category, with data saved to a local JSON file so tasks persist between sessions.

Author : Kanak Chaudhary

Project: VaultofCodes Internship - Python Programming - Assignment-4

```
"""
```

```
import json
```

```
import os
```

```
# -----
```

```
# CONSTANTS
```

```
# -----
```

```
DATA_FILE = "tasks.json"
```

```
VALID_CATEGORIES = ["Work", "Personal", "Urgent", "Other"]
```

```
# -----
```

```
# TASK CLASS
```

```
# -----
```

```
class Task:
```

```
    """
```

```
    Represents a single to-do task.
```

```
    Attributes:
```

```
        title (str) : short title of the task
```

```
        description (str) : more detail about the task
```

```
        category (str) : Work / Personal / Urgent / Other
```

```
        completed (bool) : whether the task is done
```

```
        task_id (int) : unique identifier for the task
```

```
    """
```

```
    def __init__(self, title, description, category, completed=False, task_id=None):
```

```
        self.title = title
```

```
        self.description = description
```

```
        self.category = category
```

```
        self.completed = completed
```

```
        self.task_id = task_id
```

```

def mark_completed(self):
    """Marks this task as completed."""
    self.completed = True

def to_dict(self):
    """Converts the Task object into a plain dictionary (for JSON saving)."""
    return {
        "task_id": self.task_id,
        "title": self.title,
        "description": self.description,
        "category": self.category,
        "completed": self.completed,
    }

def __str__(self):
    status = "✓ Done" if self.completed else "□ Pending"
    return (
        f"[{self.task_id}] {self.title} ({self.category}) - {status}\n"
        f"    {self.description}"
    )

```

```

# -----
# FILE HANDLING FUNCTIONS
# -----

```

```

def save_tasks(tasks):
    """Saves the full list of Task objects to the JSON data file."""
    with open(DATA_FILE, "w") as f:
        json.dump([t.to_dict() for t in tasks], f, indent=4)

def load_tasks():
    """
    Loads tasks from the JSON data file and returns a list of Task objects.
    Returns an empty list if the file doesn't exist or is corrupted.
    """
    if not os.path.exists(DATA_FILE):
        return []

    try:
        with open(DATA_FILE, "r") as f:
            raw_data = json.load(f)

```

```

        return [Task(**item) for item in raw_data]
except (json.JSONDecodeError, FileNotFoundError, TypeError):
    print("⚠ Warning: Data file was empty or corrupted. Starting fresh.")
    return []

```

```

# -----
# VALIDATION HELPERS
# -----

```

```

def get_non_empty_string(prompt):
    """Keeps asking until the user enters a non-empty string."""
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print("✗ This field cannot be empty. Please try again.")

```

```

def get_valid_category():
    """
    Lets the user pick a category from a fixed list
    (Work / Personal / Urgent / Other).
    """
    print("Choose a category:")
    for i, cat in enumerate(VALID_CATEGORIES, start=1):
        print(f" {i}. {cat}")

    while True:
        choice = input(f"Enter number (1-{len(VALID_CATEGORIES)}): ").strip()
        if choice.isdigit() and 1 <= int(choice) <= len(VALID_CATEGORIES):
            return VALID_CATEGORIES[int(choice) - 1]
        print("✗ Invalid choice. Please pick a valid number.")

```

```

def get_valid_task_id(tasks, prompt="Enter task ID: "):
    """Asks for a task ID and confirms it exists in the current task list."""
    while True:
        raw = input(prompt).strip()
        if not raw.isdigit():
            print("✗ Please enter a valid numeric ID.")
            continue
        task_id = int(raw)

```

```

for t in tasks:
    if t.task_id == task_id:
        return t
print("✗ No task found with that ID. Try again.")

```

```

# -----
# CORE FEATURES
# -----

```

```

def add_task(tasks):
    """Prompts the user for task details and adds a new Task to the list."""
    print("\n----- ADD NEW TASK -----")
    title = get_non_empty_string("Enter task title: ")
    description = get_non_empty_string("Enter task description: ")
    category = get_valid_category()

    new_id = (max((t.task_id for t in tasks), default=0)) + 1
    task = Task(title, description, category, completed=False, task_id=new_id)

    tasks.append(task)
    save_tasks(tasks)
    print(f"✅ Task added successfully! (ID: {new_id})\n")

```

```

def view_tasks(tasks):
    """
    Displays a filter menu, then shows tasks matching the chosen filter:
    All / Completed / Pending / By Category.
    """
    if not tasks:
        print("\nNo tasks yet. Add one first!\n")
        return

    print("\n----- VIEW TASKS -----")
    print("1. View all tasks")
    print("2. View completed tasks")
    print("3. View pending tasks")
    print("4. View tasks by category")
    print("5. Back to main menu")
    choice = input("Choose an option (1-5): ").strip()

    if choice == "1":
        print_task_list(tasks)

```

```

elif choice == "2":
    print_task_list([t for t in tasks if t.completed], "Completed Tasks")
elif choice == "3":
    print_task_list([t for t in tasks if not t.completed], "Pending Tasks")
elif choice == "4":
    category = get_valid_category()
    print_task_list([t for t in tasks if t.category == category], f"{category} Tasks")
elif choice == "5":
    return
else:
    print(" ✖ Invalid choice.")

```

```

def print_task_list(task_list, heading="All Tasks"):
    """Neatly prints a list of tasks, or a message if the list is empty."""
    print(f"\n--- {heading} ---")
    if not task_list:
        print("(No tasks found in this view.)")
        return
    for t in task_list:
        print(t)
        print("-" * 45)

```

```

def mark_task_completed(tasks):
    """Marks a chosen task (by ID) as completed."""
    if not tasks:
        print("\nNo tasks yet. Add one first!\n")
        return

    print_task_list(tasks)
    task = get_valid_task_id(tasks, "\nEnter the ID of the task to mark as completed: ")
    task.mark_completed()
    save_tasks(tasks)
    print(f"✅ Task '{task.title}' marked as completed!")

```

```

def edit_task(tasks):
    """Edits the title, description, or category of an existing task."""
    if not tasks:
        print("\nNo tasks yet. Add one first!\n")
        return

```

```

print_task_list(tasks)
task = get_valid_task_id(tasks, "\nEnter the ID of the task to edit: ")

print(f"\nEditing task: {task.title}")
print("Leave a field blank to keep its current value.")

new_title = input(f"New title [{task.title}]: ").strip()
if new_title:
    task.title = new_title

new_description = input(f"New description [{task.description}]: ").strip()
if new_description:
    task.description = new_description

change_category = input(f"Change category? Current: {task.category} (y/n): ").strip().lower()
if change_category == "y":
    task.category = get_valid_category()

save_tasks(tasks)
print("✅ Task updated successfully.")

```

```

def delete_task(tasks):
    """Deletes a task from the list by its ID, after confirmation."""
    if not tasks:
        print("\nNo tasks yet. Add one first!\n")
        return

    print_task_list(tasks)
    task = get_valid_task_id(tasks, "\nEnter the ID of the task to delete: ")

    confirm = input(f"Are you sure you want to delete '{task.title}'? (y/n): ").strip().lower()
    if confirm == "y":
        tasks.remove(task)
        save_tasks(tasks)
        print("✅ Task deleted successfully.")
    else:
        print("Cancelled.")

```

```

# -----
# MAIN MENU / PROGRAM LOOP
# -----

```

```

def print_main_menu():
    print("\n" + "=" * 42)
    print(" 📝 PERSONAL TO-DO LIST MANAGER 📝 ")
    print("=" * 42)
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Mark Task Completed")
    print("4. Edit Task")
    print("5. Delete Task")
    print("6. Exit")
    print("=" * 42)

def main():
    """Main program loop. Loads existing data, then shows the menu."""
    tasks = load_tasks()

    print("Welcome to your Personal To-Do List Manager!")
    if tasks:
        print(f"📁 Loaded {len(tasks)} previous task(s) from file.")
        print_task_list(tasks)
    else:
        print("📁 No previous tasks found. Starting fresh.")

    while True:
        print_main_menu()
        choice = input("Choose an option (1-6): ").strip()

        if choice == "1":
            add_task(tasks)
        elif choice == "2":
            view_tasks(tasks)
        elif choice == "3":
            mark_task_completed(tasks)
        elif choice == "4":
            edit_task(tasks)
        elif choice == "5":
            delete_task(tasks)
        elif choice == "6":
            save_tasks(tasks)
            print("\n👋 Tasks saved. Goodbye!")
            break

```


else:

print(" ✖ Invalid choice. Please select a number between 1 and 6.")

```
if __name__ == "__main__":  
    main()
```